Lab 6 Report

CSCI 5742: Cybersecurity Programming and Analytics, Spring 2026

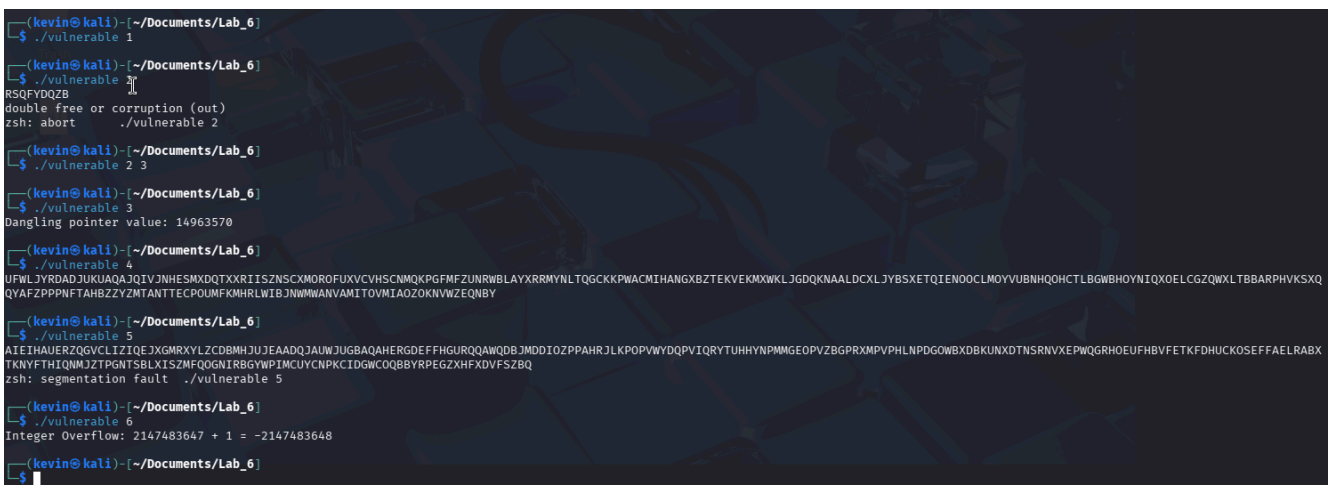
Name & Student ID: Kevin Jacob, 109750578

Task 1: Implement & Analyze Additional Vulnerabilities

1. **Commented Source Code**
   - *(Paste or attach your commented `vulnerable_program.c` code here. Provide explanations for each vulnerability.)*

2. **Program Outputs**

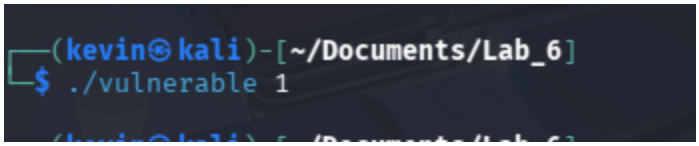
   - *(Insert 3+ screenshots of the program's output for different inputs — e.g., `./vulnerable_program 1`, `./vulnerable_program 3`, and `./vulnerable_program 5`.)*



Task 2: Out-of-Bounds Write (Valgrind)

Screenshots

1. *(Screenshot of running `./vulnerable_program 1` without Valgrind.)*



2. *(Screenshot of Valgrind output: `valgrind --tool=memcheck ./vulnerable_program 1`.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck ./vulnerable 1
==64571== Memcheck, a memory error detector
==64571== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==64571== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==64571== Command: ./vulnerable 1
==64571==
==64571== Invalid write of size 4
==64571==    at 0x4000A04: overRun (vulnerable.c:9)
==64571==    by 0x4000C63: main (vulnerable.c:73)
==64571==  Address 0x4a9f068 is 0 bytes after a block of size 40 alloc'd
==64571==    at 0x48A5498: malloc (vg_replace_malloc.c:446)
==64571==    by 0x40009F7: overRun (vulnerable.c:8)
==64571==    by 0x4000C63: main (vulnerable.c:73)
==64571==
==64571==
==64571== HEAP SUMMARY:
==64571==     in use at exit: 40 bytes in 1 blocks
==64571==   total heap usage: 1 allocs, 0 frees, 40 bytes allocated
==64571==
==64571== LEAK SUMMARY:
==64571==    definitely lost: 40 bytes in 1 blocks
==64571==    indirectly lost: 0 bytes in 0 blocks
==64571==      possibly lost: 0 bytes in 0 blocks
==64571==    still reachable: 0 bytes in 0 blocks
==64571==         suppressed: 0 bytes in 0 blocks
==64571== Rerun with --leak-check=full to see details of leaked memory
==64571==
==64571== For lists of detected and suppressed errors, rerun with: -s
==64571== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

┌──(kevin㉿kali)-[~/Documents/Lab_6]
```

3. *(Screenshot of Valgrind with `--leak-check=full`.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --leak-check=full ./vulnerable 1
==65812== Memcheck, a memory error detector
==65812== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==65812== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==65812== Command: ./vulnerable 1
==65812==
==65812== Invalid write of size 4
==65812==    at 0x4000A04: overRun (vulnerable.c:9)
==65812==    by 0x4000C63: main (vulnerable.c:73)
==65812==  Address 0x4a9f068 is 0 bytes after a block of size 40 alloc'd
==65812==    at 0x48A5498: malloc (vg_replace_malloc.c:446)
==65812==    by 0x40009F7: overRun (vulnerable.c:8)
==65812==    by 0x4000C63: main (vulnerable.c:73)
==65812==
==65812==
==65812== HEAP SUMMARY:
==65812==     in use at exit: 40 bytes in 1 blocks
==65812==   total heap usage: 1 allocs, 0 frees, 40 bytes allocated
==65812==
==65812== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==65812==    at 0x48A5498: malloc (vg_replace_malloc.c:446)
==65812==    by 0x40009F7: overRun (vulnerable.c:8)
==65812==    by 0x4000C63: main (vulnerable.c:73)
==65812==
==65812== LEAK SUMMARY:
==65812==    definitely lost: 40 bytes in 1 blocks
==65812==    indirectly lost: 0 bytes in 0 blocks
==65812==      possibly lost: 0 bytes in 0 blocks
==65812==    still reachable: 0 bytes in 0 blocks
==65812==         suppressed: 0 bytes in 0 blocks
==65812==
==65812== For lists of detected and suppressed errors, rerun with: -s
==65812== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)

┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$
```

4. *(Screenshot after fixing the `overRun` function to confirm no more errors.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --leak-check=full ./vulnerable 1
==67544== Memcheck, a memory error detector
==67544== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==67544== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==67544== Command: ./vulnerable 1
==67544==
==67544==
==67544== HEAP SUMMARY:
==67544==     in use at exit: 0 bytes in 0 blocks
==67544==   total heap usage: 1 allocs, 1 frees, 40 bytes allocated
==67544==
==67544== All heap blocks were freed -- no leaks are possible
==67544==
==67544== For lists of detected and suppressed errors, rerun with: -s
==67544== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Answers to Questions

- **1.** Why does this invalid write error happen?
  (Answer here)
  It happens because the code attempts to write to the 10th index of the array but that memory does not exist
- **2.** Why does Valgrind report an "invalid write of size 4"? What does `4` represent?
  (Answer here)
  IT represents the 4 bytes of memory used by a single int
- **3.** What is an off-by-one error? Do you see this error in the `overRun` function?
  (Answer here)
  An off by one error happens whne a loop or index targets a position that is 1 index too far past the boundary
- **4.** What is a memory leak? Explain in your own words. Do you see a memory leak in the `overRun` function?
  (Answer here)
  A memory leak happens when an allocated piece of memory is not released once it is done being used. Overrun has a leak because the memory that is alloced is not freed.
- **5.** Can errors like this occur in Java? Why or why not?
  (Answer here)
  No it cannot because Java uses automatic bounds checking and throws an exception instead of trying to write out of bounds.
- **6.** Compare the Heap Summary from normal Valgrind output vs. `--leak-check=full`. What additional details are shown?
  (Answer here)
  It provides a leak summary that categorizes memory and points to the exact line where the lost memory was originally allocated.

Updated Code for `overRun` Function

```
/* Insert your corrected overRun function here.
   Include inline comments explaining the fix. */
//    #include <stdlib.h>
// #include <stdio.h>
// #include <string.h>
// #include <time.h>
// #include <limits.h>  // Required for integer limits

// void overRun(void) {
//     int *x = malloc(10 * sizeof(int));

//     if (x == NULL) {return;}

//     x[9] = 0;  // Buffer overrun
//     free(x);
// }

// void randStringGen(int x, char* c) {
```

```
//     srand(time(NULL));
//     for (int i = 0; i < x - 1; ++i) {
//         *c = 'A' + (rand() % 26);
//         c++;
//     }
//     *c = '\0';
// }

// void bufferUnder(void) {
//     char buffer[256];
//     char *c = malloc(255 * sizeof(char));
//     randStringGen(255, c);
//     strcpy(buffer, c);  // Possible buffer overflow
//     printf("%s\n", buffer);
//     free(c);
// }

// void danglingPtr(void) {
//     int *x;
//     int *y = malloc(10 * sizeof(int));
//     x = y;
//     free(y);  // x is now a dangling pointer
//     int t = x[2];  // Accessing freed memory
//     printf("Dangling pointer value: %d\n", t);
// }

// void unInitializedPtr(void) {
//     char *buffer;
//     char *c = malloc(10 * sizeof(char));
//     randStringGen(10, c);
//     strcpy(buffer, c);  // Using an uninitialized pointer
//     printf("%s\n", buffer);
//     free(c);
//     free(buffer);
// }

// void bufferOver(void) {
//     char buffer[256];
//     char *c = malloc(260 * sizeof(char));
//     randStringGen(260, c);
//     strcpy(buffer, c);  // Buffer overflow
//     printf("%s\n", buffer);
//     free(c);
// }

// // New: Integer Overflow Vulnerability
// void integerOverflow(void) {
//     int a = INT_MAX;  // Max signed int value
//     int b = 1;
//     int result = a + b;  // Causes overflow
//     printf("Integer Overflow: %d + %d = %d\n", a, b, result);
```

```
// }

// int main(int argc, char**argv) {
//     if (argc != 2) {
//         return 0;
//     }
//     int x = atoi(argv[1]);  // Convert input to integer

//     if (x == 1) {
//         overRun();
//     } else if (x == 2) {
//         unInitializedPtr();
//     } else if (x == 3) {
//         danglingPtr();
//     } else if (x == 4) {
//         bufferUnder();
//     } else if (x == 5) {
//         bufferOver();
//     } else if (x == 6) {
//         integerOverflow();
//     }

//     return 0;
// }
// #include <stdlib.h>
// #include <stdio.h>
// #include <string.h>
// #include <time.h>
// #include <limits.h>  // Required for integer limits

// void overRun(void) {
//     int *x = malloc(10 * sizeof(int));

//     if (x == NULL) {return;}

//     x[9] = 0;  // Buffer overrun
//     free(x);
// }

// void randStringGen(int x, char* c) {
//     srand(time(NULL));
//     for (int i = 0; i < x - 1; ++i) {
//         *c = 'A' + (rand() % 26);
//         c++;
//     }
//     *c = '\0';
// }

// void bufferUnder(void) {
//     char buffer[256];
//     char *c = malloc(255 * sizeof(char));
```

```
//     randStringGen(255, c);
//     strcpy(buffer, c);  // Possible buffer overflow
//     printf("%s\n", buffer);
//     free(c);
// }

// void danglingPtr(void) {
//     int *x;
//     int *y = malloc(10 * sizeof(int));
//     x = y;
//     free(y);  // x is now a dangling pointer
//     int t = x[2];  // Accessing freed memory
//     printf("Dangling pointer value: %d\n", t);
// }

// void unInitializedPtr(void) {
//     char *buffer;
//     char *c = malloc(10 * sizeof(char));
//     randStringGen(10, c);
//     strcpy(buffer, c);  // Using an uninitialized pointer
//     printf("%s\n", buffer);
//     free(c);
//     free(buffer);
// }

// void bufferOver(void) {
//     char buffer[256];
//     char *c = malloc(260 * sizeof(char));
//     randStringGen(260, c);
//     strcpy(buffer, c);  // Buffer overflow
//     printf("%s\n", buffer);
//     free(c);
// }

// // New: Integer Overflow Vulnerability
// void integerOverflow(void) {
//     int a = INT_MAX;  // Max signed int value
//     int b = 1;
//     int result = a + b;  // Causes overflow
//     printf("Integer Overflow: %d + %d = %d\n", a, b, result);
// }

// int main(int argc, char**argv) {
//     if (argc != 2) {
//         return 0;
//     }
//     int x = atoi(argv[1]);  // Convert input to integer

//     if (x == 1) {
//         overRun();
//     } else if (x == 2) {
```

```
//         unInitializedPtr();
//     } else if (x == 3) {
//         danglingPtr();
//     } else if (x == 4) {
//         bufferUnder();
//     } else if (x == 5) {
//         bufferOver();
//     } else if (x == 6) {
//         integerOverflow();
//     }

//     return 0;
// }
```

Task 3: Uninitialized Pointer Analysis

Screenshots

1. *(Screenshot of `valgrind --tool=memcheck --leak-check=full ./vulnerable_program 2`.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full ./vulnerable 2

==71911== Memcheck, a memory error detector
==71911== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==71911== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==71911== Command: ./vulnerable 2
==71911==
==71911== Use of uninitialised value of size 8
==71911==    at 0x48AE7A0: strcpy (vg_replace_strmem.c:565)
==71911==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==71911==    by 0x4000C8F: main (vulnerable.c:79)
==71911==
==71911== Use of uninitialised value of size 8
==71911==    at 0x48AE7A8: strcpy (vg_replace_strmem.c:565)
==71911==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==71911==    by 0x4000C8F: main (vulnerable.c:79)
==71911==
==71911== Use of uninitialised value of size 8
==71911==    at 0x48AE7B8: strcpy (vg_replace_strmem.c:565)
==71911==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==71911==    by 0x4000C8F: main (vulnerable.c:79)
==71911==
==71911== Conditional jump or move depends on uninitialised value(s)
==71911==    at 0x48AE7C8: is_overlap (vg_replace_strmem.c:136)
==71911==    by 0x48AE7C8: strcpy (vg_replace_strmem.c:565)
==71911==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==71911==    by 0x4000C8F: main (vulnerable.c:79)
==71911==
==71911== Conditional jump or move depends on uninitialised value(s)
==71911==    at 0x48AE7D0: is_overlap (vg_replace_strmem.c:145)
==71911==    by 0x48AE7D0: is_overlap (vg_replace_strmem.c:132)
==71911==    by 0x48AE7D0: strcpy (vg_replace_strmem.c:565)
==71911==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==71911==    by 0x4000C8F: main (vulnerable.c:79)
==71911==
```

2. *(Screenshot with `--track-origins=yes` for more detail.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 2

==72652== Memcheck, a memory error detector
==72652== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==72652== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==72652== Command: ./vulnerable 2
==72652==
==72652== Use of uninitialised value of size 8
==72652==    at 0x48AE7A0: strcpy (vg_replace_strmem.c:565)
==72652==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==72652==    by 0x4000C8F: main (vulnerable.c:79)
==72652==  Uninitialised value was created by a stack allocation
==72652==    at 0x4000B50: unInitializedPtr (vulnerable.c:43)
==72652==
==72652== Use of uninitialised value of size 8
==72652==    at 0x48AE7A8: strcpy (vg_replace_strmem.c:565)
==72652==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==72652==    by 0x4000C8F: main (vulnerable.c:79)
==72652==  Uninitialised value was created by a stack allocation
==72652==    at 0x4000B50: unInitializedPtr (vulnerable.c:43)
==72652==
==72652== Use of uninitialised value of size 8
==72652==    at 0x48AE7B8: strcpy (vg_replace_strmem.c:565)
==72652==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==72652==    by 0x4000C8F: main (vulnerable.c:79)
==72652==  Uninitialised value was created by a stack allocation
==72652==    at 0x4000B50: unInitializedPtr (vulnerable.c:43)
==72652==
==72652== Conditional jump or move depends on uninitialised value(s)
==72652==    at 0x48AE7C8: is_overlap (vg_replace_strmem.c:136)
==72652==    by 0x48AE7C8: strcpy (vg_replace_strmem.c:565)
==72652==    by 0x4000B7B: unInitializedPtr (vulnerable.c:47)
==72652==    by 0x4000C8F: main (vulnerable.c:79)
==72652==  Uninitialised value was created by a stack allocation
==72652==    at 0x4000B50: unInitializedPtr (vulnerable.c:43)
==72652==
==72652== Conditional jump or move depends on uninitialised value(s)
==72652==    at 0x48AE7D0: is_overlap (vg_replace_strmem.c:145)
==72652==    by 0x48AE7D0: is_overlap (vg_replace_strmem.c:132)
==72652==    by 0x48AE7D0: strcpy (vg_replace_strmem.c:565)
```

3. *(Screenshot of fixed function showing no more uninitialized pointer usage issues.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 2

==10052== Memcheck, a memory error detector
==10052== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==10052== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==10052== Command: ./vulnerable 2
==10052==
VTMRRFMWA
==10052==
==10052== HEAP SUMMARY:
==10052==     in use at exit: 0 bytes in 0 blocks
==10052==   total heap usage: 3 allocs, 3 frees, 1,044 bytes allocated
==10052==
==10052== All heap blocks were freed -- no leaks are possible
==10052==
==10052== For lists of detected and suppressed errors, rerun with: -s
==10052== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$
```

Answers to Questions

- **7.** Where is the memory problem occurring? What does Valgrind report?
  (Answer here)
  The memory problem occurs at the strcpy operation. Valgrind show is as a use of uninitialized value because the buffer pointer was declared but it was never assigned a valid memory address.
- **8.** What is an uninitialized pointer? How could it be exploited?
  (Answer here)
  An uninitialized pointer contains a random garbage value from the stack, it could be xploited to overwite sensitive data.
- **9.** What is the difference between a `NULL` pointer and an uninitialized pointer?
  (Answer here)
  A NULL pointer is set to 0 and it causes crash if it is used. An uninitialized pointer contains random data and has unpredictable behavior.
- **10.** What specifically in the code do you believe caused the uninitialized pointer usage?
  (Answer here)
  THe char *buffer delcares a pointer but does not allocate memory for it. Then strcpy tries to write data to whatever random address buffer happens to contain
- **11.** What additional detail does `--track-origins=yes` provide?
  (Answer here)
  It allows valgrind to identify the specific line of code where the uninitialized variable was first created. This allows you to trace the error back to the source.
- **12.** "Use of uninitialized value of size 8" — what does the `8` refer to?
  (Answer here)
  The 8 refers to the 8 bytes required to store a memory address.

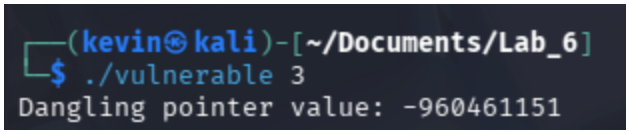
Updated Code for `unInitializedPtr` Function

```
/* Insert your corrected unInitializedPtr function here.
   Include inline comments explaining the fix.
   void unInitializedPtr(void) {
    char *buffer = malloc(10 * sizeof(char));

    // Safety check for memory allocation
    if (buffer == NULL) {
        return;
    }

    char *c = malloc(10 * sizeof(char));
    if (c == NULL) {
        free(buffer);
        return;
    }

    randStringGen(10, c);

    // Now that 'buffer' points to a valid memory block, strcpy is safe.
    strcpy(buffer, c);

    printf("%s\n", buffer);

    free(c);
    free(buffer);
}
   */
```

Task 4: Dangling Pointer Analysis

Screenshots

1. *(Screenshot of `./vulnerable_program 3` without Valgrind — note behavior.)*



2. *(Screenshot of Valgrind output: `valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable_program 3`.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 3

==16461== Memcheck, a memory error detector
==16461== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==16461== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==16461== Command: ./vulnerable 3
==16461==
==16461== Invalid read of size 4
==16461==    at 0x4000B2C: danglingPtr (vulnerable.c:39)
==16461==    by 0x4000CD7: main (vulnerable.c:93)
==16461==  Address 0x4a9f048 is 8 bytes inside a block of size 40 free'd
==16461==    at 0x48A8538: free (vg_replace_malloc.c:989)
==16461==    by 0x4000B27: danglingPtr (vulnerable.c:38)
==16461==    by 0x4000CD7: main (vulnerable.c:93)
==16461==  Block was alloc'd at
==16461==    at 0x48A5498: malloc (vg_replace_malloc.c:446)
==16461==    by 0x4000B13: danglingPtr (vulnerable.c:36)
==16461==    by 0x4000CD7: main (vulnerable.c:93)
==16461==
Dangling pointer value: 0
==16461==
==16461== HEAP SUMMARY:
==16461==     in use at exit: 0 bytes in 0 blocks
==16461==   total heap usage: 2 allocs, 2 frees, 1,064 bytes allocated
==16461==
==16461== All heap blocks were freed -- no leaks are possible
==16461==
==16461== For lists of detected and suppressed errors, rerun with: -s
==16461== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)

┌──(kevin㉿kali)-[~/Documents/Lab_6]
```

3. *(Screenshot after fixing `danglingPtr`, showing no error.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 3

==18247== Memcheck, a memory error detector
==18247== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==18247== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==18247== Command: ./vulnerable 3
==18247==
Valid pointer value: 42
==18247==
==18247== HEAP SUMMARY:
==18247==     in use at exit: 0 bytes in 0 blocks
==18247==   total heap usage: 2 allocs, 2 frees, 1,064 bytes allocated
==18247==
==18247== All heap blocks were freed -- no leaks are possible
==18247==
==18247== For lists of detected and suppressed errors, rerun with: -s
==18247== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Answers to Questions

- **13.** What is the potential issue in the `danglingPtr` function?
  (Answer here)
  The issue lies in the attempted use of pointer x after the memory (y) has already been freed.
- **14.** How could a dangling pointer be exploited?
  (Answer here)

A dangling pointer points to memory the program no longer has. This can be used by an attacker to reallocate the memory to a different variablem and use that to read or corrupt new data.

- **15.** What does Valgrind report about the freed memory usage?
  (Answer here)
  It reports an invalid read size of 4 since the memory does not exist anymore for the program to use.
- **16.** Why does Valgrind possibly show no final "heap error" even though it's a dangerous bug?
  (Answer here)
  Becuase all the memory that was allocated was freed, which means there is technically no leak.

Updated Code for `danglingPtr` Function

```
/* Insert your corrected danglingPtr function here.
   Include inline comments explaining the fix.
   void danglingPtr(void) {
    int *x;
    int *y = malloc(10 * sizeof(int));

    // Safety check for allocation
    if (y == NULL) {
        return;
    }

    x = y;

    Access the memory while it is still valid.
    x[2] = 42;
    int t = x[2];
    printf("Valid pointer value: %d\n", t);

    // Free the memory only when you are completely finished with it.
    free(y);

    // Set pointers to NULL after freeing to prevent accidental "use-after-
free".
    y = NULL;
    x = NULL;
}

   */
```

Task 5: Buffer Overflows Analysis

Screenshots

- **For `bufferUnder` (Input 4):**

1. *(Screenshot of Valgrind output with `./vulnerable_program 4`.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 4

==27964== Memcheck, a memory error detector
==27964== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==27964== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==27964== Command: ./vulnerable 4
==27964==
HXXVFDUSQGNNKPYIWOIPMBJGCMQYHBPOYOMGRGAHONVYDVGBLQQXRBGVPYTWZKNAZZGSHGAVVXVASBCGTUDMWLINKBMJOBLPCRHJZHGXGBXBCBHVWMJUYRHIVTTJUFYWYFHAPNXVOWYSYFPWSZQQQZALVTWRAUQZCXBTNAOBWNWWULSMMKEFMGSHA
PYALOBPOCIBCXEBMCXGOSVCEBHQIAXKRYMCMOSCSADXZKYNMXWCRRFWUOOCQOOHMBKAPE
==27964==
==27964== HEAP SUMMARY:
==27964==     in use at exit: 0 bytes in 0 blocks
==27964==   total heap usage: 2 allocs, 2 frees, 1,279 bytes allocated
==27964==
==27964== All heap blocks were freed -- no leaks are possible
==27964==
==27964== For lists of detected and suppressed errors, rerun with: -s
==27964== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$
```

- **For `bufferOver` (Input 5):**

2. *(Screenshot of Valgrind output with `./vulnerable_program 5` — if any overflow detected.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 4

==27964== Memcheck, a memory error detector
==27964== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==27964== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==27964== Command: ./vulnerable 4
==27964==
HXXVFDUSQGNNKPYIWOIPMBJGCMQYHBPOYOMGRGAHONVYDVGBLQQXRBGVPYTWZKNAZZGSHGAVVXVASBCGTUDMWLINKBMJOBLPCRHJZHGXGBXBCBHVWMJUYRHIVTTJUFYWYFHAPNXVOWYSYFPWSZQQQZALVTWRAUQZCXBTNAOBWNWWULSMMKEFMGSHA
PYALOBPOCIBCXEBMCXGOSVCEBHQIAXKRYMCMOSCSADXZKYNMXWCRRFWUOOCQOOHMBKAPE
==27964==
==27964== HEAP SUMMARY:
==27964==     in use at exit: 0 bytes in 0 blocks
==27964==   total heap usage: 2 allocs, 2 frees, 1,279 bytes allocated
==27964==
==27964== All heap blocks were freed -- no leaks are possible
==27964==
==27964== For lists of detected and suppressed errors, rerun with: -s
==27964== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$
```

3. *(Screenshot of AddressSanitizer detection using `./vulnerable_program2 5`.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ ./vulnerable2 5
=================================================================
==29435==ERROR: AddressSanitizer: stack-buffer-overflow on address 0xfbffa0200120 at pc 0xffffa2373a2c bp 0xffffcf8bbad0 sp 0xffffcf8bbb20
WRITE of size 260 at 0xfbffa0200120 thread T0
    #0 0xffffa2373a28 in strcpy ../../../../src/libsanitizer/asan/asan_interceptors.cpp:568
    #1 0xaaaabbda144c in bufferOver /home/kevin/Documents/Lab_6/vulnerable.c:86
    #2 0xaaaabbda15c0 in main /home/kevin/Documents/Lab_6/vulnerable.c:114
    #3 0xffffa2142518 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #4 0xffffa21425f8 in __libc_start_main_impl ../csu/libc-start.c:360
    #5 0xaaaabbda0dec in _start (/home/kevin/Documents/Lab_6/vulnerable2+0x0dec) (BuildId: 536d7bbeca3447d8e3b95014c5393c516f6ca0be)

Address 0xfbffa0200120 is located in stack of thread T0 at offset 288 in frame
    #0 0xaaaabbda13a8 in bufferOver /home/kevin/Documents/Lab_6/vulnerable.c:82

  This frame has 1 object(s):
    [32, 288) 'buffer' (line 83) <== Memory access at offset 288 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow /home/kevin/Documents/Lab_6/vulnerable.c:86 in bufferOver
Shadow bytes around the buggy address:
  0xfbffa01ffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa01fff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa01fff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200000: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0xfbffa0200100: 00 00 00 00[f3]f3 f3 f3 f3 f3 f3 f3 00 00 00 00
  0xfbffa0200180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200380: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
```

4. *(Screenshot after fixing `bufferOver`, no errors remain.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ ./vulnerable2 5
=================================================================
==29435==ERROR: AddressSanitizer: stack-buffer-overflow on address 0xfbffa0200120 at pc 0xffffa2373a2c bp 0xffffcf8bbad0 sp 0xffffcf8bbb20
WRITE of size 260 at 0xfbffa0200120 thread T0
    #0 0xffffa2373a28 in strcpy ../../../../src/libsanitizer/asan/asan_interceptors.cpp:568
    #1 0xaaaabbda144c in bufferOver /home/kevin/Documents/Lab_6/vulnerable.c:86
    #2 0xaaaabbda15c0 in main /home/kevin/Documents/Lab_6/vulnerable.c:114
    #3 0xffffa2142518 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
    #4 0xffffa21425f8 in __libc_start_main_impl ../csu/libc-start.c:360
    #5 0xaaaabbda0dec in _start (/home/kevin/Documents/Lab_6/vulnerable2+0xdec) (BuildId: 536d7bbeca3447d8e3b95014c5393c516f6ca0be)

Address 0xfbffa0200120 is located in stack of thread T0 at offset 288 in frame
    #0 0xaaaabbda13a8 in bufferOver /home/kevin/Documents/Lab_6/vulnerable.c:82

  This frame has 1 object(s):
    [32, 288) 'buffer' (line 83) <= Memory access at offset 288 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow /home/kevin/Documents/Lab_6/vulnerable.c:86 in bufferOver
Shadow bytes around the buggy address:
  0xfbffa01ffe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa01fff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa01fff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200000: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0xfbffa0200100: 00 00 00 00[f3]f3 f3 f3 f3 f3 f3 f3 00 00 00 00
  0xfbffa0200180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200300: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  0xfbffa0200380: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
  Addressable:           00
  Partially addressable: 01 02 03 04 05 06 07
  Heap left redzone:       fa
  Freed heap region:       fd
  Stack left redzone:      f1
  Stack mid redzone:       f2
  Stack right redzone:     f3
  Stack after return:      f5
  Stack use after scope:   f8
  Global redzone:          f9
  Global init order:       f6
  Poisoned by user:        f7
  Container overflow:      fc
```

Answers to Questions

- **(Regarding `bufferUnder`, Input 4)**
  - **15.** Do you see errors in the Valgrind output?
    (Answer here)
    No Valgrind does not report any errors.
  - **16.** After reading the code, do you expect errors? Why/why not?
    (Answer here)
    No because the function generates a 255 character which should fit perfectly into the 256 buffer.
- **(Regarding `bufferOver`, Input 5)**
  - **17.** Do you expect an error here? Why?
    (Answer here)
    Yes because the code attempts to copy 260 characters into a buffer that only holds 256 bytes.
  - **18.** Does Valgrind detect it? If so, what is reported?
    (Answer here)
    No valgrind usually does not detect buffer overflows
  - **19.** Why does Valgrind sometimes struggle to detect this kind of buffer overflow? *(Answer here)*
    Valgrind primarily checks the heap memory and struggles to detect overflows which are usually stack based errors.
- **(Valgrind vs. Other Tools)**

- **20.** List two additional Valgrind tools besides `memcheck`.
  (Answer here)
  Massif, and helgrind
- **21.** How could these other tools detect errors that `memcheck` misses?
  (Answer here)
  Massif is a heap profiles that identifies inefficient use of memory. Helgrind detects synchronization errors in multithreaded programs, wwhich are logical uses rather than just memory access violations.

AddressSanitizer Findings

- **22.** What errors does AddressSanitizer report for input `5`?
  (Answer here)
  AddressSanitizer reports a buffer overflow error and crashes the program
- **23.** Where in the code does it say the error occurs?
  (Answer here)
  It says that the error occurs at the strcpy line within the bufferOver function
- **24.** How does AddressSanitizer compare to Valgrind in detecting buffer overflows?
  (Answer here)
  AddressSanitizer is much better at catching stack based errors like a buffer overflow because analyzes the code at compile time to place restrictive zones around the stack variables.

Updated Code for `bufferOver` Function

```
/* Insert your corrected bufferOver function here.
   Include inline comments explaining the fix.

   void bufferOver(void) {
    // A statically-sized array on the stack
    char buffer[256];

    // Allocate 260 bytes in the heap
    char *c = malloc(260 * sizeof(char));
    if (c == NULL) {
        return;
    }

    // Generate a 260-character string
    randStringGen(260, c);

    // Use strncpy instead of strcpy to limit the copy size.
    // We use 255 to leave room for the null terminator ('\0').
    strncpy(buffer, c, sizeof(buffer) - 1);

    // Manually ensure the string is null-terminated.
    // strncpy does not append a null terminator if the limit is reached.
    buffer[255] = '\0';
```

```
    printf("%s\n", buffer);

    //  Free the heap memory to prevent a leak.
    free(c);
}*/
```

Task 6: Integer Overflow Analysis

Screenshots

1. *(Screenshot of `./vulnerable_program 6` showing normal run — note any incorrect result.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ ./vulnerable 6
Integer Overflow: 2147483647 + 1 = -2147483648
```

2. *(Screenshot of `valgrind --tool=memcheck ... ./vulnerable_program 6` showing whether it detects overflow.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 6

==32703== Memcheck, a memory error detector
==32703== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==32703== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==32703== Command: ./vulnerable 6
==32703==
Integer Overflow: 2147483647 + 1 = -2147483648
==32703==
==32703== HEAP SUMMARY:
==32703==     in use at exit: 0 bytes in 0 blocks
==32703==   total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==32703==
==32703== All heap blocks were freed -- no leaks are possible
==32703==
==32703== For lists of detected and suppressed errors, rerun with: -s
==32703== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

3. *(Screenshot of UBSan detection: `./vulnerable_program2 6`.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ ./vulnerable2 6
vulnerable.c:95:9: runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
Integer Overflow: 2147483647 + 1 = -2147483648
```

4. *(Screenshot of fixed function, showing no more overflow vulnerability.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ valgrind --tool=memcheck --leak-check=full --track-origins=yes ./vulnerable 6

==44435== Memcheck, a memory error detector
==44435== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==44435== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==44435== Command: ./vulnerable 6
==44435==
Integer Overflow detected! Operation aborted to prevent undefined behavior.
==44435==
==44435== HEAP SUMMARY:
==44435==     in use at exit: 0 bytes in 0 blocks
==44435==   total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==44435==
==44435== All heap blocks were freed -- no leaks are possible
==44435==
==44435== For lists of detected and suppressed errors, rerun with: -s
==44435== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Answers to Questions

- **25.** Why does the overflow occur at `UINT_MAX + 1`?
  (Answer here)
  The issue occurs because adding 1 to the the max value of a signed integer causes the bits to wrap around and result in a negative number.
- **26.** What are common security risks of integer overflows, and how might attackers exploit them?
  (Answer here) Integer overflows can cause logic errors and attackers can exploit them to bypass boundary checks. They can allocate a tiny buffer and copy a massive payload into it and trigger a buffer overflow.
- **27.** Does Valgrind report the integer overflow? If not, why?
  (Answer here) No it does not because it tracks memory erorrs, not logical errors
- **28.** Does UBSan report an error?
  (Answer here)
  Yes it does
- **29.** Where in the code does UBSan say the overflow occurs?
  (Answer here)
  It occurs at the line int result = a + b
- **30.** Compare UBSan's detection to Valgrind's.
  (Answer here)
  UBSan is specifically designed to catch mathematical and logical errors via a compile time scan. Valgrind would not be able to catch an error like this.

Updated Code for `integerOverflow` Function

```
/* Insert your corrected integerOverflow function here.
   Include inline comments explaining the fix.

   void integerOverflow(void) {
```

```
    int a = INT_MAX;  // Max signed int value (typically 2,147,483,647)
    int b = 1;
    int result;

    // Implement a boundary check before performing the addition.
    // We check if 'a' is greater than (INT_MAX - b).
    // If it is, adding 'b' to 'a' will definitely cause an overflow.
    if (a > INT_MAX - b) {
        printf("Integer Overflow detected! Operation aborted to prevent
undefined behavior.\n");
    } else {
        result = a + b;
        printf("Result: %d\n", result);
    }
}*/
```

Task 7: Static Analysis with Flawfinder

Screenshots

1. *(Screenshot of `flawfinder vulnerable_program.c` output.)*

```
┌──(kevin㉿kali)-[~/Documents/Lab_6]
└─$ flawfinder vulnerable.c
Flawfinder version 2.0.19, (C) 2001-2019 David A. Wheeler.
Number of rules (primarily dangerous function names) in C/C++ ruleset: 222
Examining vulnerable.c

FINAL RESULTS:

vulnerable.c:29:  [4] (buffer) strcpy:
  Does not check for buffer overflows when copying to destination [MS-banned]
  (CWE-120). Consider using snprintf, strcpy_s, or strlcpy (warning: strncpy
  easily misused).
vulnerable.c:76:  [4] (buffer) strcpy:
  Does not check for buffer overflows when copying to destination [MS-banned]
  (CWE-120). Consider using snprintf, strcpy_s, or strlcpy (warning: strncpy
  easily misused).
vulnerable.c:86:  [4] (buffer) strcpy:
  Does not check for buffer overflows when copying to destination [MS-banned]
  (CWE-120). Consider using snprintf, strcpy_s, or strlcpy (warning: strncpy
  easily misused).
vulnerable.c:17:  [3] (random) srand:
  This function is not sufficiently random for security-related functions
  such as key and nonce creation (CWE-327). Use a more secure technique for
  acquiring random values.
vulnerable.c:26:  [2] (buffer) char:
  Statically-sized arrays can be improperly restricted, leading to potential
  overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use
  functions that limit length, or ensure that the size is larger than the
  maximum possible length.
vulnerable.c:83:  [2] (buffer) char:
  Statically-sized arrays can be improperly restricted, leading to potential
  overflows or other issues (CWE-119!/CWE-120). Perform bounds checking, use
  functions that limit length, or ensure that the size is larger than the
  maximum possible length.
vulnerable.c:112:  [2] (integer) atoi:
  Unless checked, the resulting number can exceed the expected range
  (CWE-190). If source untrusted, check both minimum and maximum, even if the
  input had no minus sign (large numbers can roll over into negative number;
  consider saving to an unsigned value if that is intended).

ANALYSIS SUMMARY:

Hits = 7
Lines analyzed = 129 in approximately 0.00 seconds (48771 lines/second)
```

Answers to Questions

- **31.** Differentiate static vs. dynamic analysis of source code.
  (Answer here)
  Static analysis looks at the code without execution and identifies any potential errors. Dynamic analysis involvdes catching the issue at runtime.
- **32.** How do static analysis tools like Flawfinder differ from dynamic tools (Valgrind, AddressSanitizer)?
  (Answer here)
  Flawfinder is able to flag any functions that could cause any potential erorrs by simply viewing the raw text of the code. Valgrind and other dynamic tools require the code to be compiled in order to catch any errors.

Flawfinder Vulnerabilities

- **33.** `strcpy` issues
  - Location, risk level, CWE classification, and prevention.
    (Answers here)
  - Occurs under bufferUnder, unInitializedPtr, and bufferOver
  - High risk
  - CWE-120
  - Use bounds checking functions instead
- **34.** `srand` usage (weak randomness)
  - Why is it a concern, relevant CWE, safer alternatives.
    (Answers here)
  - randStringGen
  - It is a concern because using the current system time as a seed makes the randomly generated numbers highly predictable
  - CWE-337
  - Use cryptographically secure random number generators
- **35.** Statically-sized arrays
  - Where used, security risks, relevant CWE, safer approaches.
    (Answers here)
  - bufferUnder, bufferOver
  - If an input excdeeds the maximum size, it will overwrite adjacent memory
  - CWE-119
  - By dynamically allocating memory, or strictly enforcing boundary checks *(Paste or summarize key parts of the Flawfinder output. Explain any false positives or unaddressed concerns.)*

Lab 6: Summary & Reflections

Key Takeaways from Lab 6

(Summarize your main findings, what you learned, and any challenges faced during the lab.)

This lab explores the use of various analysis tools, both dynamic and static. Static analysis tools like flawfinder are effective for catching potential logical errors, and known vulnerabilities before the code is even compiled. Dynamic analysis tools require the execution of the program in order to catch any memory errors. Most memory corruption vulnerabilities originate from poor memory management and lack of boundary enforcement. Using bounded functions and enforcing strict checks before arithmetic operations, initializing all points to NULL, and releasing them after use drastically reduce any memory related vulnerabilities.